Group Assignment: Concurrent Linear Algebra Engine & CI

Master Course in Programming — Practical Workflow

Riccardo Rossi Ruben Zorrilla Carlos Roig

October 23, 2026

Assignment Overview

Core Objectives

- Practice branch management & concurrent development.
- Build a custom Linear Algebra library (`mini_linalg`).
- Set up Continuous Integration (CI) with GitHub Actions.
- Resolve multi-branch merge conflicts manually.
- Refactor monolithic code into modular archives.

Team Requirements

- Teams of 3 students.
- 6 distinct feature branches created concurrently from `main`.
- Mandatory peer PR reviews before merging.
- Enforced passing tests (`pytest`) and linting (`flake8`).

Initial Repository Architecture (main)

Directory Layout

```
mini_linalg/
| __init__.py
| core.py
| test_core.py
 README.md
```

Starter core.py

```python
class Vector:
    def __init__(self, data: list[float]):
        self.data = list(data)

    def __repr__(self) -> str:
        return f"Vector({self.data})"

class Matrix:
    def __init__(self, data: list[list[float]]):
        self.data = [list(r) for r in data]

    def __repr__(self) -> str:
        return f"Matrix({self.data})"
```

Branch Allocation & Responsibilities

Branch Name	Assignee	Target Files	Key Tasks
`feature/ci-pipeline`	Team	`.github/workflows/ci.yml`	Set up Python matrix, `pytest`, and `flake8`
`feature/pure-python-ops`	Student 1	`core.py`, `test_core.py`	Implement Matrix-Vector & Matrix-Matrix products
`feature/pytest-suite`	Student 2	`tests/test_core.py`	Full test coverage, dimension mismatch error handling
`feature/numpy-vectorization`	Student 2	`core.py`	Vectorize `Vector` & `Matrix` internals using NumPy
`feature/batched-operations`	Student 3	`core.py`, `test_core.py`	Add batch matrix/vector operations
`feature/modular-refactor`	Team	`src/mini_linalg/*`	Split `core.py` into submodules and remove `core.py`

Final Modular Refactoring Phase

Goal: Modularization & Cleanup

After all feature branches are integrated into `main`, the team must collaboratively refactor the codebase to transition away from the monolithic `core.py` file.

Submodule Breakdown

- Create `implementation/vector.py` (`Vector` class).
- Create `implementation/matrix.py` (`Matrix` class).
- Create `implementation/__init__.py` (empty file).

Removal & Backwards Compatibility

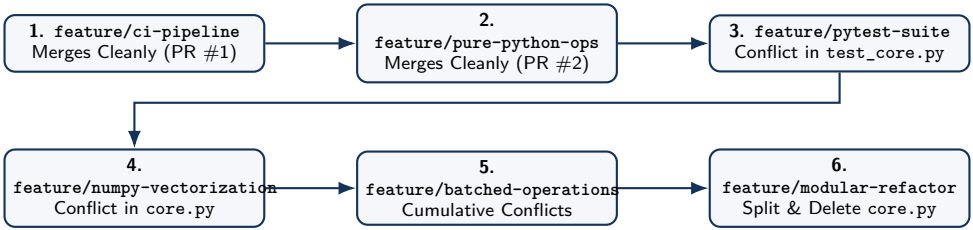
- Expose imports inside `__init__.py`.
- Completely delete the original `core.py` file.
- Ensure all tests run with 100% pass rate without referencing `core.py`.

Refactored Architecture (main)

Directory Layout

```
mini_linalg/
| __init__.py
| implementation/
|       | __init__.py
|       | vector.py
|       matrix.py
| test.py
 README.md
```

Sequential Integration Workflow



Local Merge Conflict Resolution Protocol

Standard Protocol for Conflict Branches

```
# 1. Switch to feature branch & fetch latest main
git checkout main
git pull
git checkout  [...here the branch name ... ]
git merge main

# 2. Resolve conflict markers (<<<<<<<, =======, >>>>>>>) in your editor
# 3. Validate locally before committing
pytest
flake8 src tests

# 4. Finalize merge and update Pull Request
git add  [..here the list of files to add ...]
git commit -m "fix: resolve merge conflicts with main"
git push
```

Summary & Expectations

Technical Requirements

- All branches MUST branch off initial `main`.
- Pure Python $\rightarrow$ NumPy vectorization.
- Continuous Integration active for all PRs.
- Clean deletion of `core.py` in final refactor.

Collaboration Rules

- No direct pushes to `main`.
- Require at least 1 peer approval per PR.
- Conflicts resolved locally on feature branches.
- All commits must maintain passing test builds.